Project Report

ON

"SENTIVISION: IMAGE-BASED SENTIMENT INTELLIGENCE"

Submitted to

School of Electronics Engineering

BACHELOR'S DEGREE IN

Electronics & Computer Science Engineering

BY

ABHIPRIYA MUKHERJEE - 2330206

SHREYAM KUMAR TIWARI - 2330267

RISHITA DHAR - 2330330

NATASHA BAG - 2330242

School of Electronics Engineering

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

BHUBANESWAR, ODISHA - 751024

November 2026

ABSTRACT

SENTIVISION is a web application and command-line tool that reads opinions from screenshots. A user uploads an image of product reviews or comments. The system cleans the image, extracts the text with EasyOCR, splits it into separate comments and classifies each one as negative, neutral or positive using the pretrained cardiffnlp/twitter-roberta-base-sentiment-latest model. Results appear in a React dashboard with confidence scores and charts, and the user can correct OCR mistakes and classify again. A FastAPI backend saves every analysis in a SQLite database. A separate evaluation tool tests the model on the TweetEval benchmark and reports accuracy, precision, recall, F1 and a confusion matrix. No model was trained for this project. The work lies in the pipeline that joins OCR with sentiment analysis, and in the testing and documentation around it.

Keywords: sentiment analysis, OCR, EasyOCR, RoBERTa, FastAPI, React, TweetEval

TABLE OF CONTENTS

1. Title and Group Details..... 5
2. Problem Statement and Objectives.....6
2.1 Problem statement.....6
2.2 Motivation.....6
2.3 Objectives..... 6
2.4 Expected outcomes..... 7
2.5 Scope and assumptions..... 7
3. Proposed Methodology and Processing Pipeline..... 8
3.1 Step-by-step explanation.....8
3.2 Input, intermediate data and output..... 9
3.3 The stages in more detail..... 9
3.4 Design decisions..... 10
4. Models and Tools Used..... 11
4.1 Pretrained models..... 11
4.2 Dataset..... 11
4.3 Libraries and frameworks..... 11
4.4 Implementation environment.....12
4.5 Why these tools.....12
5. Implementation and Usability.....13
5.1 How it was built.....13
5.2 Running the application.....13
5.3 Practical uses.....14
5.4 Pages of the web app..... 14
5.5 REST API..... 15
5.6 Testing..... 15
6. Results and Sample Outputs..... 16
6.1 Sample output: single text prediction..... 16
6.2 Sample output: screenshot analysis..... 16
6.3 Performance evaluation..... 18
6.4 Observations..... 19
6.5 Limitations.....20
7. Conclusion and Future Scope.....21
7.1 Summary.....21
7.2 Possible improvements..... 21
7.3 Final remarks..... 21
8. References and GitHub Repository Link.....22
8.1 GitHub repository..... 22
8.2 Models, datasets and software..... 22
8.3 Papers.....22

LIST OF FIGURES

Figure 1. SENTIVISION processing pipeline (six stages).....8
Figure 2. Input screenshot of two product reviews..... 16
Figure 3. Batch Analysis results page..... 17
Figure 4. Overview dashboard..... 17
Figure 5. Settings page..... 18

1. Title and Group Details

Project title: SENTIVISION, Image-Based Sentiment Intelligence

Course / Institution: Electronics & Computer Science Engineering, KIIT University, Bhubaneswar

Academic year: 2026–27

SENTIVISION reads a screenshot of customer reviews or comments, pulls out the text, and tells you whether each comment is negative, neutral or positive. The table below lists the group and who did what. Please replace the bracketed fields with the real details.

Member	Roll number	Contribution
Abhipriya Mukherjee	2330206	OCR pipeline, image preprocessing, comment segmentation
Shreyam Kumar Tiwari	2330267	Sentiment model integration, evaluation CLI, metrics
Rishita Dhar	2330330	FastAPI backend, database, automated tests
Natasha Bag	2330242	React dashboard, charts, report, presentation

2. Problem Statement and Objectives

2.1 Problem statement

People share opinions about products as screenshots all the time. They grab a page of reviews, a comment thread or a social media post and send it around as an image. The text inside is locked up. You can't search it, count it or run it through a sentiment tool until someone types it out by hand.

Most sentiment analysis tools expect clean text as input. This project handles the missing step: take the image, recover the text, split it into separate comments and classify each one.

Two separate technologies have to work together here. Optical character recognition (OCR) turns pixels into characters. Sentiment analysis then decides whether the words express a negative, neutral or positive opinion. Each one is well studied on its own. Chaining them is where things get messy, because a small OCR mistake can flip the meaning of a sentence, and a screenshot rarely holds just one comment. The project is built around handling that handoff sensibly.

2.2 Motivation

- Businesses read a lot of feedback that only exists as images, such as app store reviews, chat screenshots and survey captures.
- Reading these by hand is slow and the results vary from person to person.
- Pretrained transformer models now give good sentiment results without any training of our own, so the main engineering work is getting clean text out of the picture.

2.3 Objectives

1. Accept a JPG, JPEG or PNG screenshot and extract the text with OCR.
2. Split the extracted text into individual comments instead of one long block.
3. Classify every comment as negative, neutral or positive, with a confidence score.
4. Show the results in a simple web dashboard that lets the user fix OCR mistakes and re-run the classification.
5. Measure the sentiment model honestly on a labelled public dataset using a command-line tool.
6. Keep a history of past analyses so results can be reviewed later.

2.4 Expected outcomes

- A working web app and CLI that go from screenshot to sentiment in one step.
- Per-comment labels with class probabilities, plus summary charts.
- Evaluation reports (accuracy, precision, recall, F1 and a confusion matrix) that anyone can reproduce.
- A documented GitHub repository that another student can clone and run.

2.5 Scope and assumptions

The system works on English text only and handles three sentiment classes. It does not train or fine-tune any model, and it does not try to judge whether a review is fake. It assumes the screenshot is reasonably legible, meaning the text is readable by a person without zooming in heavily. Everything runs locally, so no image or comment is sent to an outside service after the models have been downloaded once.

3. Proposed Methodology and Processing Pipeline

The system is a straight pipeline. A screenshot goes in at one end and a list of labelled comments comes out the other. The block diagram below shows the six stages.

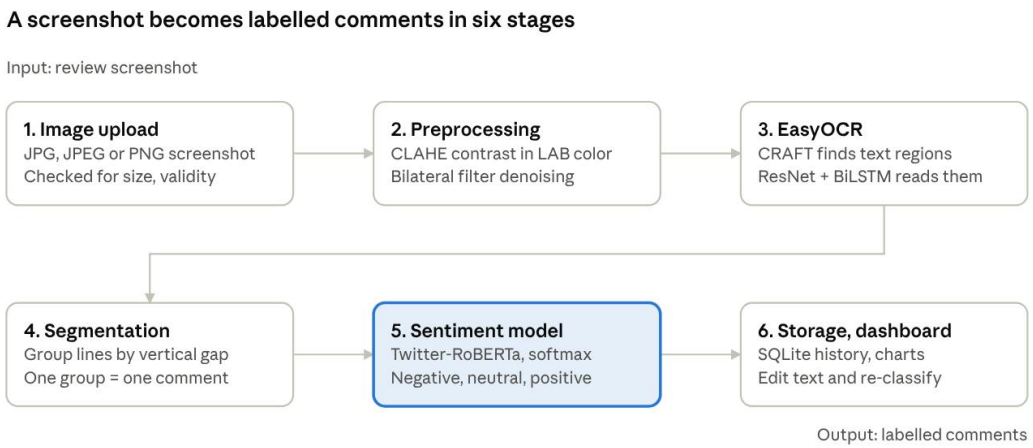


Figure 1. SENTIVISION processing pipeline (six stages).

Read the top row left to right, then continue along the bottom row. The sentiment model (highlighted) is the only machine learning step we run for classification.

3.1 Step-by-step explanation

1. **Image upload and validation.** The user uploads a JPG, JPEG or PNG file. The backend checks that the file opens properly and stays within the size limit. Corrupted files are rejected with a clear error. The batch mode accepts up to 20 screenshots and processes them one after another so memory use stays low.
2. **Preprocessing.** Screenshots often have low contrast or compression noise. The image is converted to the LAB color space and CLAHE (contrast limited adaptive histogram equalization) is applied to the lightness channel. This makes faint text easier to read without changing colors. A bilateral filter then removes noise while keeping the edges of letters sharp.
3. **Text detection and recognition.** EasyOCR runs in two parts. The CRAFT detector finds where text sits in the image. A ResNet and BiLSTM recognizer with CTC decoding then reads the characters in each region. For every piece of text we get a bounding box, the recognized string and a confidence value.
4. **Comment segmentation.** OCR returns lines, not comments. We sort the lines from top to bottom and group those that sit close together vertically. A larger gap starts a new group. Each group becomes one comment, so two reviews on the same screenshot are no longer merged into one.
5. **Sentiment classification.** Each comment is passed to the Twitter-RoBERTa model. The model produces a score for each class and a softmax turns the scores into probabilities. The class with the highest probability is the predicted

label, and its probability is the confidence. Label names are read from the model's own configuration, so we never assume an order.

6. **Storage and display.** Results are saved in a local SQLite database and sent to the web dashboard. The dashboard shows each comment in a card with its label, confidence and class probabilities. If the OCR made a mistake, the user can edit the text in the card and the comment is classified again straight away.

3.2 Input, intermediate data and output

Stage	Input	Intermediate result	Output
Upload	Screenshot file	Validated image	Image bytes
Preprocessing	Image bytes	Contrast-enhanced, denoised image	Clean image
OCR	Clean image	Bounding boxes, text lines, confidences	List of text lines
Segmentation	Text lines with positions	Lines grouped by vertical distance	List of comments
Sentiment	Comment text	Class scores, then probabilities	Label and confidence per comment
Storage and display	Labelled comments	Database record	Dashboard cards and charts

3.3 The stages in more detail

Preprocessing. The contrast step is applied only to the lightness channel of the LAB image. That way the text gets sharper without the colors of the screenshot shifting. The bilateral filter smooths noise but leaves the edges of letters alone, which matters because OCR depends on clean edges.

OCR and the confidence cutoff. EasyOCR gives every detected line a confidence score between 0 and 1. Lines below the cutoff are dropped so that stray marks and icons don't end up as fake comments. The default cutoff is 0.40 and can be changed with `OCR_CONFIDENCE_THRESHOLD` in the `.env` file. The recognition language is English.

Segmentation. The detected lines are sorted from top to bottom. The code then looks at how far each line sits from the one above it. Lines that are close together belong to the same comment. When the gap becomes large, a new comment starts. This is a purely geometric rule, so it needs no extra model and runs instantly, but it depends on the screenshot having visible spacing between comments.

Sentiment scoring. The model returns one raw score per class. A softmax turns those scores into probabilities that add up to 1:

$$P(c_i) = exp(z_i) / [\, exp(z_1) + exp(z_2) + exp(z_3) \,]$$

Here z is the score for class i. The label with the highest probability is the prediction, and that probability is shown as the confidence. Inference runs without gradient tracking, so the same text always gives the same answer.

Editing and re-scoring. Every comment card has an editable text box. If the OCR misread a word, the user fixes it and the comment is classified again straight away. The corrected text is what gets saved to the history.

3.4 Design decisions

- **Pretrained models only.** A strong pretrained classifier exists already, so effort went into the pipeline around it instead.
- **Labels read from the model.** The label order comes from the model's own configuration at runtime. Some sentiment models list classes in a different order, and a hardcoded order would silently swap results.
- **One image at a time in batch mode.** Screenshots are processed in sequence, which keeps memory use low on an ordinary laptop.
- **Mocked models in tests.** The tests don't load the real models, so they finish quickly and work offline.

4. Models and Tools Used

4.1 Pretrained models

We did not train any model. Both the OCR engine and the sentiment classifier are pretrained and used as they are.

Task	Model	Source
Sentiment classification	cardiffnlp/twitter-roberta-base-sentiment-latest	Hugging Face model page
Text detection	CRAFT, inside EasyOCR	EasyOCR on GitHub
Text recognition	ResNet + BiLSTM + CTC, inside EasyOCR	EasyOCR on GitHub

About the sentiment model. It is a RoBERTa-base transformer with 12 layers, 768 hidden units, 12 attention heads and about 125 million parameters. It was pretrained on roughly 124 million tweets and then fine-tuned for sentiment. It predicts three classes: negative, neutral and positive. Because it learned from social media text, it handles short, informal comments well, which suits review screenshots.

About the OCR engine. EasyOCR is an open source library. CRAFT finds text regions, and the recognizer reads each region. Language weights (about 100 MB) download on the first run and are cached.

How the transformer reads a comment. The text is split into tokens, and each token is turned into a vector. Twelve layers of self-attention then let every word look at every other word in the comment, so the model can tell that "not good" means something different from "good". A small classification layer on top produces the three class scores. Training on a very large amount of tweets before fine-tuning is what lets it cope with short, informal and sometimes messy text.

4.2 Dataset

For evaluation we use the TweetEval sentiment test split, a public benchmark of human-labelled tweets with the same three classes. It is available on Hugging Face as cardiffnlp/tweet_eval. The CLI downloads it and saves a metadata file with the source, license and sample count.

4.3 Libraries and frameworks

Area	Tools
Frontend	React 18, TypeScript, Vite, Tailwind CSS, Recharts, Framer Motion, Axios, Lucide icons

Area	Tools
Backend	FastAPI, Pydantic v2, Uvicorn, SQLAlchemy with SQLite
Machine learning	PyTorch, Hugging Face Transformers
Computer vision	OpenCV (headless), Pillow
Evaluation	Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn
Command line	Typer, Rich, Hugging Face datasets
Testing	pytest

4.4 Implementation environment

- Python 3.11 or newer, with a virtual environment
- Node.js 18 or newer and npm 9 or newer
- Git
- Runs on CPU. If a CUDA GPU is found, PyTorch uses it automatically. Setting `DEVICE=cpu` in the `.env` file forces CPU.
- First run downloads about 100 MB of OCR weights and about 500 MB for the RoBERTa model, then works from the cache.

4.5 Why these tools

Tool	Reason for choosing it
EasyOCR	Free, runs locally, installs with pip and returns bounding boxes and confidence scores, which the segmentation step needs
Twitter-RoBERTa	Public, well documented, three classes, and trained on short informal text similar to comments
FastAPI	Quick to write, validates requests automatically and generates interactive API docs
React with Vite	Fast development server and easy charting for the dashboard
SQLite	No separate database server to install, so the project runs on any laptop
Typer	Turns plain Python functions into a clean command-line tool

5. Implementation and Usability

5.1 How it was built

The project has four parts that talk to each other in a simple way.

- **Backend.** A FastAPI app exposes the endpoints for health checks, text analysis, image analysis, batch analysis, evaluation and history. The OCR and sentiment models load once when the server starts, so each request only pays for inference.
- **Frontend.** A React and TypeScript app built with Vite. It sends uploads to the backend with Axios and draws the results with Recharts: a donut chart for the sentiment split, grouped bars for comparison and a trend line over past analyses. It has light and dark themes.
- **CLI.** A Typer program in `cli/cli.py` that works without the web app. It predicts sentiment for a single sentence or a CSV file, downloads public datasets and runs evaluations.
- **Database.** SQLite through SQLAlchemy stores every analysis so the history page can list and delete past records.

The tests use mocked models so they run fast and offline. They cover OCR preprocessing, segmentation, sentiment inference, the API routes and the evaluator maths.

5.2 Running the application

Setup (once)

```
git clone https://github.com/enort3533/sentivision.git
cd sentivision
python3.11 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
```

Start the backend

```
cd backend
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

The API runs at `http://localhost:8000` and the interactive docs are at `http://localhost:8000/docs`.

Start the frontend (in a second terminal)

```
cd sentivision/frontend
npm install
npm run dev
```

Open `http://localhost:5173` in a browser. The dev server forwards API calls to the backend.

Use the command line

```
# one sentence
python cli/cli.py predict --text "The product quality is superb! Highly
recommend."

# download the benchmark and evaluate
python cli/cli.py download-dataset --source tweet_eval --split test
python cli/cli.py evaluate --input data/datasets/tweet_eval_test.csv --
text-column text --label-column sentiment
```

Run the tests

```
pytest tests/ -v
```

If port 8000 is busy, start Uvicorn on `--port 8001` and update the proxy in `frontend/vite.config.ts`.

5.3 Practical uses

- Product teams can drop in screenshots of app store or e-commerce reviews and get a quick sentiment split.
- Support teams can check the mood of a chat or social media thread that was shared as an image.
- Students and researchers can use the evaluation lab to test the model on their own labelled CSV files.
- Anyone can fix OCR slips right in the card, so a small reading error doesn't spoil a result.

The interface is deliberately plain: upload, look at the cards, edit if needed. No setup beyond installing the dependencies is required, and nothing leaves the user's machine.

5.4 Pages of the web app

The sidebar has six pages. There is also a light and dark theme switch at the bottom.

Page	What it does
Overview	Shows whether the backend, OCR engine and model are ready, the total number of comments analyzed, and two charts: the overall sentiment split and the split per screenshot
Analyze Screenshot	Upload one image and see a card for every detected comment, with an editable text box and class probabilities
Batch Analysis	Upload up to 20 images at once, watch them process one by one, and download the combined results as a CSV file
Evaluation Lab	Upload a labelled CSV file and get accuracy, precision, recall, F1 scores and a confusion matrix
History	Lists past analyses so they can be reopened or deleted
Settings	Shows the model name, the device in use, the label mapping and the OCR settings

5.5 REST API

The frontend talks to the backend through these routes. They can also be tried directly from the interactive docs page.

Method	Route	Purpose
GET	/api/health	Health check with model and OCR status
GET	/api/model/info	Model name, labels and device
POST	/api/analyze/text	Classify plain text
POST	/api/analyze/image	Screenshot to OCR, segmentation and sentiment
POST	/api/analyze/batch	Same as above for up to 20 screenshots
POST	/api/evaluate	Upload a labelled CSV and get metrics
GET	/api/history	List saved analyses
DELETE	/api/history/{id}	Remove one saved analysis

5.6 Testing

The automated tests run with pytest and use mocked models, so they don't need a download or a GPU.

Test file	What it checks
test_sentiment.py	Positive, negative, neutral, empty and very long text, label mapping and batch inference
test_ocr.py	Valid images, corrupted files, size limits, contrast enhancement and line grouping
test_api.py	The health, text analysis, model info and history routes
test_evaluator.py	Accuracy, macro F1, confusion matrix, custom label names and empty datasets

6. Results and Sample Outputs

6.1 Sample output: single text prediction

Input given to the CLI: *The product quality is superb! Highly recommend.*

```
Predicted Sentiment: POSITIVE
Confidence: 96.8%

Class Probabilities:
  Negative      0.8%
  Neutral       2.4%
  Positive     96.8%
```

The model is very sure about this one, and the three probabilities add up to 100%.

6.2 Sample output: screenshot analysis

For a screenshot, the dashboard shows one card per detected comment. Each card holds the OCR text, the predicted label, the confidence and the three class probabilities. A summary donut chart shows the share of each sentiment across the whole image.

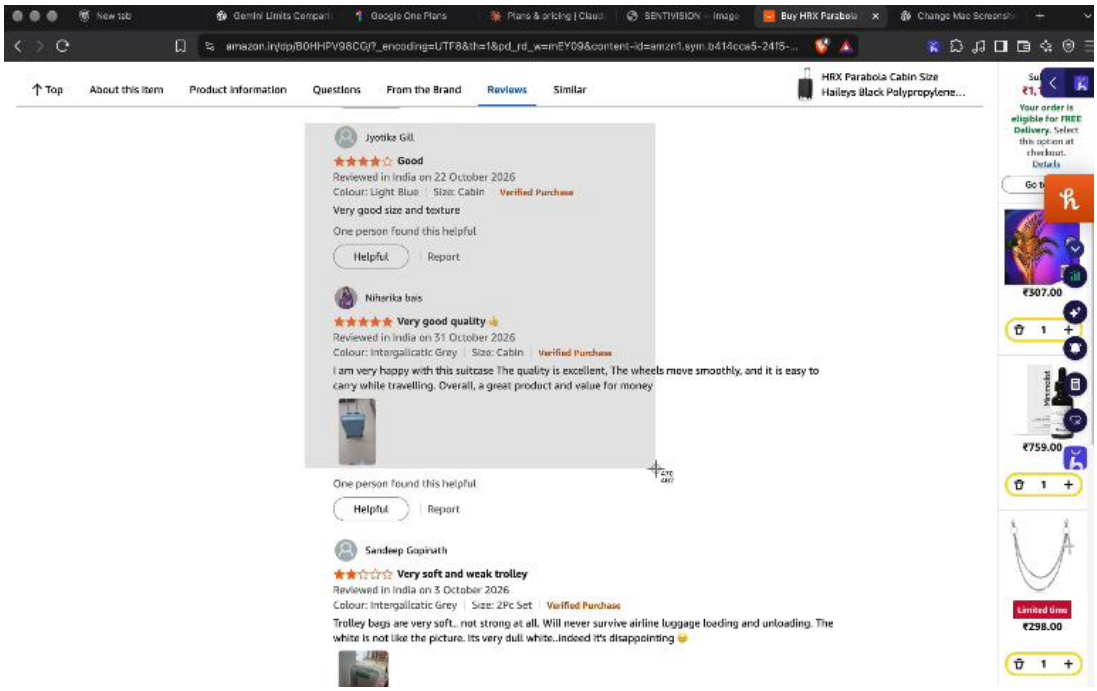


Figure 2. Input: a screenshot of two product reviews is captured from an Amazon listing.

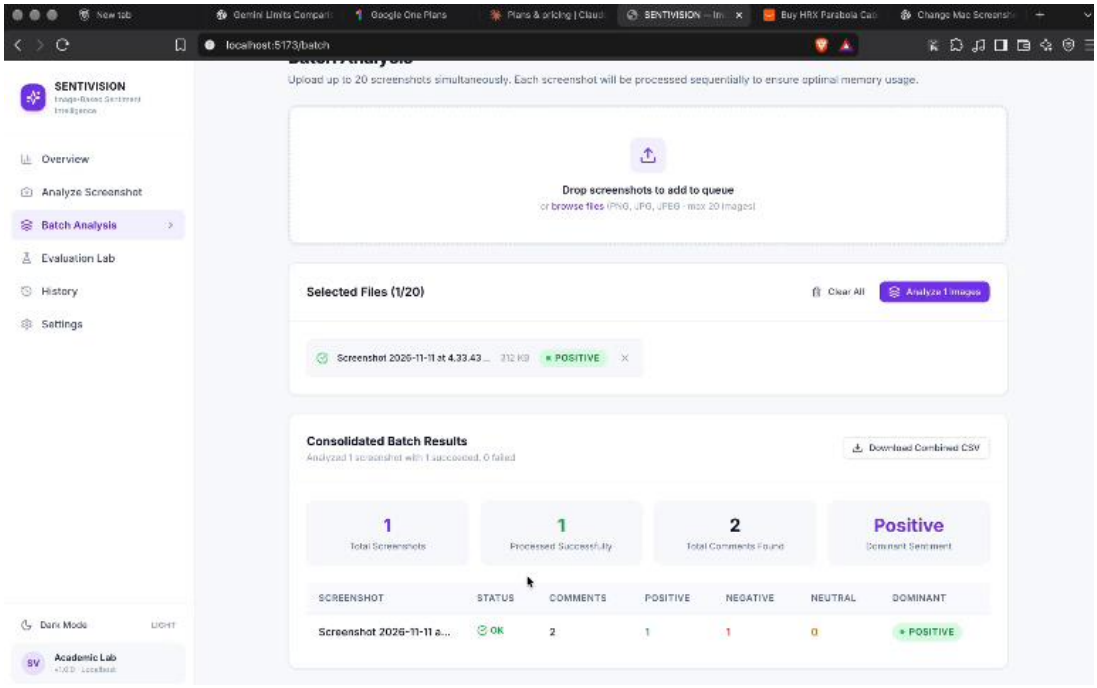


Figure 3. Batch Analysis: the uploaded screenshot gives 2 comments, 1 positive and 1 negative.

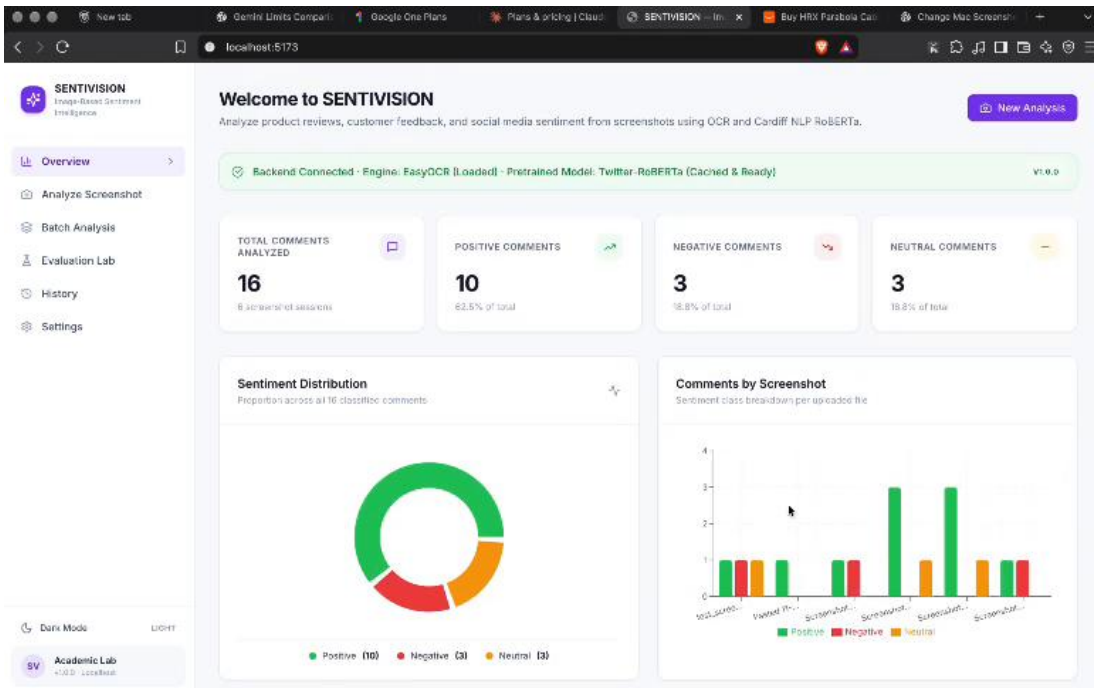


Figure 4. Overview dashboard: 16 comments from 8 screenshot sessions, split 10 positive, 3 negative and 3 neutral.

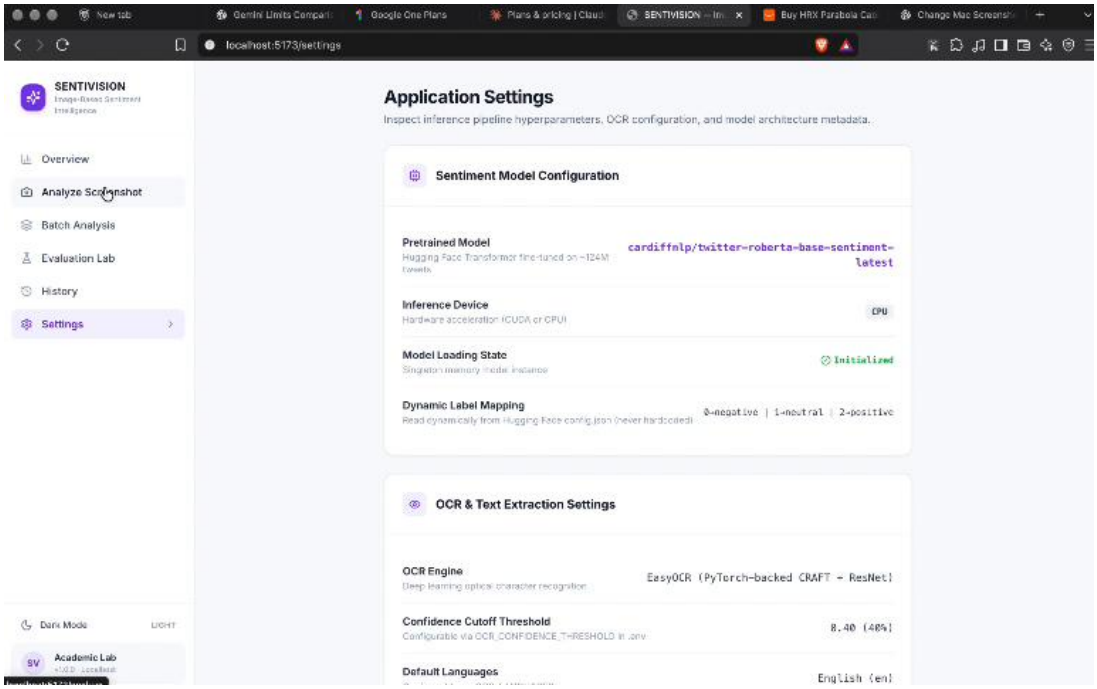


Figure 5. Settings page: the pretrained model, CPU inference, label mapping (0 negative, 1 neutral, 2 positive) and an OCR confidence cutoff of 0.40 for English text.

Source	Comments detected	Negative	Neutral	Positive
Amazon reviews screenshot (Figure 3)	2	1	0	1
All 8 sessions in history (Figure 4)	16	3	3	10

Add your own sample images and their outputs to the `samples/` folder in the repository and quote two or three here.

6.3 Performance evaluation

The model was evaluated on the TweetEval sentiment test split with the CLI. Each run writes a timestamped folder under `reports/` with `metrics.json`, a confusion matrix image and a list of misclassified samples. Copy the numbers from your own run into this table.

Metric	Value
Samples evaluated	[from metrics.json]
Accuracy	[from metrics.json]
Macro F1	[from metrics.json]
Weighted F1	[from metrics.json]
Macro precision	[from metrics.json]
Macro recall	[from metrics.json]

[Insert confusion_matrix.png here]

Macro F1 is the headline number. It averages the F1 score of the three classes with equal weight, so a large neutral class can't hide weak results on the others.

What the metrics mean. Precision answers the question: of the comments the model called positive, how many really were? Recall asks the opposite: of the truly positive comments, how many did the model find? F1 combines the two into one number:

$$F1 = (2 \times Precision \times Recall) / (Precision + Recall)$$

The confusion matrix lists true labels against predicted labels. The diagonal shows the correct predictions, and any number off the diagonal shows which two classes get mixed up.

Files written by each evaluation run.

File	Contents
metrics.json	Accuracy, macro and weighted F1, precision and recall
classification_report.txt	Precision, recall and support for each class
confusion_matrix.png	Heatmap of the confusion matrix
predictions.csv	True label, predicted label and confidence for every sample
misclassified_samples.csv	Only the wrong predictions, for studying failures
evaluation_summary.txt	A readable summary of the run

6.4 Observations

- Clear, strongly worded comments are classified with high confidence.
- Neutral is the hardest class. Mixed or factual comments often sit between neutral and one of the other two labels.
- OCR quality drives the final result. When text is read wrongly, the sentiment can change, which is why the editable cards matter.
- Splitting by line spacing works well on screenshots with clear gaps between comments.

Please check these against your own runs and adjust the wording if your results differ.

Note on the demo run. In Figure 3 the batch summary shows one positive and one negative comment for a screenshot of two reviews rated 4 and 5 stars. [Open the comment cards for this image and write here which text was labelled negative and why, for example an OCR error or a line that is not a review.]

6.5 Limitations

- The sentiment model was fine-tuned on tweets. Performance on long product reviews or technical text may be lower.
- Only three classes are predicted. There is no sarcasm detection and no aspect-level sentiment (for example, good battery but poor screen).
- OCR can struggle with stylised fonts, very small text, low-resolution images and emoji.
- Comment segmentation relies on vertical spacing. Tightly packed or multi-column layouts can be split in the wrong place.
- The model works on English text only.
- Large batches are processed one image at a time, so they can be slow on a CPU.

7. Conclusion and Future Scope

7.1 Summary

SENTIVISION turns a screenshot of reviews into labelled, searchable sentiment data. It combines image cleanup, EasyOCR, a simple line-grouping step and a pretrained Twitter-RoBERTa classifier, and wraps them in a web dashboard and an evaluation CLI. We reused strong pretrained models and put our effort into the parts around them: preprocessing, segmentation, an editable review step, honest evaluation on a public benchmark and a clean, tested codebase that others can run.

Challenges and lessons. Most of the difficulty sat between the two models, not inside them. OCR output is noisy, comments run together, and sentiment models are sensitive to the exact words they receive. Letting the user correct the text before it is scored turned out to be a simple fix for a lot of these problems. We also learned to avoid assumptions about model internals, such as the order of the labels, and to read them from the model's configuration instead. Finally, writing tests with mocked models made it easy to change the code without waiting for large models to load.

7.2 Possible improvements

- **Domain fine-tuning.** Fine-tune the sentiment model on product review data to close the gap between tweets and reviews.
- **More languages.** Add multilingual OCR and a multilingual sentiment model.
- **Aspect-level sentiment.** Report sentiment per topic, such as price, delivery or quality, instead of one label per comment.
- **Better segmentation.** Use layout analysis to handle multi-column pages and chat bubbles.
- **Stronger OCR on hard images.** Try a different OCR engine or add deskewing and upscaling for blurry screenshots.
- **Faster batches.** Run batch images in parallel on a GPU machine.
- **Export.** Let users download results as CSV or PDF reports.

7.3 Final remarks

SENTIVISION shows that a useful tool can be built by connecting well-chosen pretrained models with careful engineering. It meets the objectives set at the start: it reads text from screenshots, separates comments, classifies them, shows the results clearly and reports its accuracy on a public dataset. The limitations are real, and they point to clear next steps. We hope the repository helps others who need to get opinions out of images.

8. References and GitHub Repository Link

8.1 GitHub repository

Repository: https://github.com/enort3533/sentivision

8.2 Models, datasets and software

1. Cardiff NLP. *twitter-roberta-base-sentiment-latest* (model). https://huggingface.co/cardiffnlp/twitter-roberta-base-sentiment-latest
2. Cardiff NLP. *TweetEval* (dataset). https://huggingface.co/datasets/cardiffnlp/tweet_eval
3. JaidedAI. *EasyOCR*. https://github.com/JaidedAI/EasyOCR
4. Hugging Face. *Transformers library*. https://huggingface.co/docs/transformers
5. FastAPI documentation. https://fastapi.tiangolo.com

8.3 Papers

1. Liu, Y. et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692.
2. Barbieri, F. et al. (2020). TweetEval: Unified Benchmark and Comparative Evaluation for Tweet Classification. Findings of EMNLP 2020.
3. Loureiro, D. et al. (2022). TimeLMs: Diachronic Language Models from Twitter. ACL 2022, System Demonstrations.
4. Baek, Y. et al. (2019). Character Region Awareness for Text Detection (CRAFT). CVPR 2019.
5. Zuiderveld, K. (1994). Contrast Limited Adaptive Histogram Equalization. In Graphics Gems IV.
6. Wolf, T. et al. (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP 2020, System Demonstrations.